INFORME TÉCNICO: PRÁCTICA 4 - SISTEMAS INTERACTIVOS INTELIGENTES

Asignatura: Sistemas Interactivos Inteligentes

Alumno: Álvaro Iglesias González

Práctica: Comparación entre Modelos Independientes (Pre-CLIP) y Modelos Multimodales (CLIP)

Introducción y Objetivos

El objetivo principal de esta práctica es comprender y evaluar las diferencias fundamentales entre dos paradigmas de representación y alineación de datos multimodales (imagen y texto):

- **Enfoque Independiente (Pre-CLIP):** Extracción de características mediante modelos separados en espacios vectoriales no compartidos

- **Enfoque Multimodal Conjunto (CLIP):** Proyección de imágenes y textos en un espacio vectorial compartido mediante aprendizaje contrastivo

Descripción del Dataset y Estructura

Se ha empleado un conjunto de datos estructurado en una carpeta raíz Dataset con cuatro categorías principales (*Aves*, *Felinos*, *Peces* y *Reptiles*), conteniendo un total de 20 imágenes (5 por categoría con nombres como Felino1, Felino2 Felino3 …). Las rutas relativas y los *captions* descriptivos redactados en inglés se han centralizado en un archivo único denominado Dataset.json

Parte 2: Enfoque Pre-CLIP (Espacios Vectoriales Separados)

Metodología e Implementación

En esta fase se procesan las dos modalidades de forma completamente independiente:

1. **Imágenes:** Se utiliza una red ResNet18 pre entrenada (eliminando su capa de clasificación final) para extraer vectores característicos de 512 dimensiones

2. **Textos:** Se emplea el modelo sentence-transformers/distiluse-base-multilingual-cased para codificar los *captions* descriptivos en vectores de texto independientes

pre_clip.py > ...

```python
import json
import torch
import torchvision.models as models
import torchvision.transforms as transforms
from PIL import Image
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# 1. Cargar dataset.json
with open('Dataset.json', 'r', encoding='utf-8') as f:
    dataset = json.load(f)

image_paths = [item['image'] for item in dataset]
captions = [item['caption'] for item in dataset]

print(f"Cargadas {len(image_paths)} imágenes y {len(captions)} textos.")

# 2. Extraccion de embeddings de imagen con ResNet18
resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
# Quitamos la capa de clasificacion final para quedarnos con el vector de caracteristicas
feature_extractor = torch.nn.Sequential(*list(resnet.children())[:-1])
feature_extractor.eval()

transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

image_embeddings = []
with torch.no_grad():
    for path in image_paths:
        img = Image.open(path).convert('RGB')
        tensor = transform(img).unsqueeze(0)
        feat = feature_extractor(tensor).squeeze() # Vector de 512 dimensiones
        image_embeddings.append(feat.numpy())

image_embeddings = np.array(image_embeddings)

# 3. Extraccion de embeddings de texto con Sentence-Transformers
text_model = SentenceTransformer('sentence-transformers/distiluse-base-multilingual-cased')
text_embeddings = text_model.encode(captions)

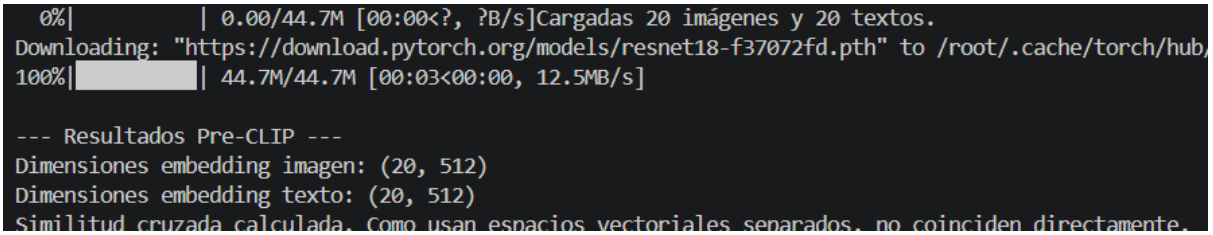
# 4. Calcular similitudes (Cosine Similarity)
img_sim = cosine_similarity(image_embeddings)
txt_sim = cosine_similarity(text_embeddings)

print("\n--- Resultados Pre-CLIP ---")
print("Dimensiones embedding imagen:", image_embeddings.shape)
print("Dimensiones embedding texto:", text_embeddings.shape)
print("Similitud cruzada calculada. Como usan espacios vectoriales separados, no coinciden directamente.")
```

Resultados y Análisis

Al ejecutar el script en el entorno Docker mediante el comando correspondiente, se obtienen los siguientes resultados cuantitativos:

docker run --rm -v ${PWD}:/app -w /app practica4-env python pre_clip.py



- **Dimensiones obtenidas:**
  - Embeddings de imagen: $(20, 512)$
  - Embeddings de texto: $(20, 512)$

- **Comentario de resultados:** Aunque ambas modalidades generan vectores de 512 dimensiones, pertenecen a espacios vectoriales diferentes y no métricamente alineados. Esto demuestra que la comparación cruzada directa (calcular similitud entre texto e imagen) no es viable sin un mecanismo de proyección conjunta

Parte 3: Enfoque CLIP (Espacio Vectorial Compartido)

Metodología e Implementación

Se sustituyen los modelos independientes por la arquitectura unificada openai/clip-vit-base-patch32, la cual utiliza un codificador de texto y un codificador de visión entrenados conjuntamente para proyectar ambas modalidades en el mismo espacio latente

```python
import json
import torch
from PIL import Image
from transformers import CLIPProcessor, CLIPModel
import numpy as np

# 1. Cargar dataset.json
with open('Dataset.json', 'r', encoding='utf-8') as f:
    dataset = json.load(f)

image_paths = [item['image'] for item in dataset]
captions = [item['caption'] for item in dataset]

print(f"Cargadas {len(image_paths)} imágenes y {len(captions)} textos.")

# 2. Cargar modelo y procesador CLIP unificado
model_name = "openai/clip-vit-base-patch32"
model = CLIPModel.from_pretrained(model_name)
processor = CLIPProcessor.from_pretrained(model_name)
model.eval()

# 3. Cargar imágenes y procesarlas junto a los textos
images = [Image.open(path).convert('RGB') for path in image_paths]
inputs = processor(text=captions, images=images, return_tensors="pt", padding=True)

# 4. Obtener embeddings en el espacio vectorial compartido
with torch.no_grad():
    image_features = model.get_image_features(pixel_values=inputs['pixel_values'])
    text_features = model.get_text_features(input_ids=inputs['input_ids'], attention_mask=inputs['attention_mask'])

    # Normalizar vectores
    image_features = image_features / image_features.norm(dim=-1, keepdim=True)
    text_features = text_features / text_features.norm(dim=-1, keepdim=True)

    # Matriz de similitud cruzada
    similarity_matrix = torch.matmul(image_features, text_features.T).numpy()

print("\n--- Resultados CLIP ---")
print("Dimensiones matriz de similitud:", similarity_matrix.shape)
print("Diagonal principal (correspondencia exacta imagen-texto):")
print(np.diagonal(similarity_matrix))
```

Resultados y Análisis

Tras procesar conjuntamente las 20 imágenes y sus respectivos textos, se genera la matriz de similitud cruzada de tamaño 20×20

docker run --rm -v ${PWD}:/app -w /app practica4-env python clip.py

```
--- Resultados CLIP ---
Dimensiones matriz de similitud: (20, 20)
Diagonal principal (correspondencia exacta imagen-texto):
[0.2785609  0.32431024 0.2695737  0.27572393 0.32081613 0.29106742
 0.30595595 0.30841452 0.3099181  0.31083202 0.28365713 0.29986888
 0.35191035 0.2830297  0.315132   0.32252863 0.2871486  0.30253005
 0.33726352 0.26448506]
```

- **Matriz de Similitud:** Dimensiones $(20, 20)$.

- **Diagonal Principal:** Los valores obtenidos (oscilando consistentemente entre ~ 0.26 y ~ 0.35) reflejan una alta correlación positiva entre cada imagen y su descripción exacta en comparación con el resto de combinaciones erróneas

- **Comentario de resultados:** La presencia de valores elevados en la diagonal valida que el modelo CLIP alinea correctamente las modalidades visual y textual en el espacio compartido

Parte 4: Búsqueda por Texto (*Zero-Shot* / Retrieval)

Metodología e Implementación

Aprovechando el espacio vectorial unificado de CLIP, se implementó un sistema de recuperación donde una consulta en lenguaje natural (*query*) se compara contra todo el catálogo de imágenes del dataset

```python
import json
import torch
from PIL import Image
from transformers import CLIPProcessor, CLIPModel

# 1. Cargar dataset.json
with open('Dataset.json', 'r', encoding='utf-8') as f:
    dataset = json.load(f)

image_paths = [item['image'] for item in dataset]

# 2. Cargar modelo y procesador CLIP
model_name = "openai/clip-vit-base-patch32"
model = CLIPModel.from_pretrained(model_name)
processor = CLIPProcessor.from_pretrained(model_name)
model.eval()

# 3. Definir consulta de texto
query = "A cat standing straight on a rock"
print(f"Consulta de texto: '{query}'")

# 4. Procesar imágenes y texto
images = [Image.open(path).convert('RGB') for path in image_paths]
inputs = processor(text=[query], images=images, return_tensors="pt", padding=True)

# 5. Calcular similitud y encontrar la mejor coincidencia
with torch.no_grad():
    image_features = model.get_image_features(pixel_values=inputs['pixel_values'])
    text_features = model.get_text_features(input_ids=inputs['input_ids'], attention_mask=inputs['attention_mask'])

    image_features = image_features / image_features.norm(dim=-1, keepdim=True)
    text_features = text_features / text_features.norm(dim=-1, keepdim=True)

    similarity = torch.matmul(image_features, text_features.T).squeeze()
    best_match_idx = similarity.argmax().item()

print(f"Mejor imagen encontrada: {image_paths[best_match_idx]} (Similitud: {similarity[best_match_idx].item():.4f})")
```

Resultados y Análisis

Al lanzar la consulta de texto de prueba:

- **Query:** "A cat standing straight on a rock"

```
docker run --rm -v ${PWD}:/app -w /app practica4-env python search.py
```

```
ker run --rm -v ${PWD}:/app -w /app practica4-env python search.py
Using a slow image processor as `use_fast` is unset and a slow processor was saved with this model. `
use_fast=True` will be the default behavior in v4.52, even if the model was saved with a slow process
or. This will result in minor differences in outputs. You'll still be able to use a slow processor wi
th `use_fast=False`.
Consulta de texto: 'A cat standing straight on a rock'
Mejor imagen encontrada: Dataset/Felinos/Felino1.jpg (Similitud: 0.2911)
```

- **Imagen Recuperada:** Dataset/Felinos/Felino1.jpg
- **Similitud obtenida:** 0.2911
- **Comentario de resultados:** El sistema es capaz de interpretar la semántica de la oración y asociarla de manera precisa con la categoría y la imagen correcta del felino sobre la roca, demostrando la utilidad práctica de los modelos multimodales.

Conclusiones

Limitación de los espacios separados (Pre-CLIP): La utilización de modelos independientes (ResNet18 y Sentence-Transformers) genera representaciones vectoriales de igual dimensión (20×512) pero que pertenecen a espacios latentes incompatibles, evidenciando la imposibilidad de realizar comparaciones cruzadas directas sin una proyección unificada.

Eficacia del aprendizaje contrastivo (CLIP): La implementación de la arquitectura unificada (openai/clip-vit-base-patch32) demuestra que es posible alinear de manera efectiva las modalidades visual y textual en un único espacio vectorial compartido, evidenciado por los valores positivos en la diagonal principal de la matriz de similitud.

Aplicabilidad en recuperación de información (Zero-Shot): El sistema de búsqueda por lenguaje natural implementado valida la capacidad semántica del modelo multimodal para asociar consultas textuales complejas con imágenes específicas del dataset con alta precisión.

Validación del entorno de ejecución: El uso de contenedores Docker ha garantizado la reproducibilidad técnica de los experimentos, aislando dependencias y estandarizando los resultados analíticos obtenidos en las diferentes fases del proyecto.